

UNIVERSITÉ IBN TOFAÏL — KÉNITRA
ÉCOLE NATIONALE DES SCIENCES APPLIQUÉES DE KÉNITRA
FILIÈRE : INGÉNIERIE DES RÉSEAUX ET DES SYSTÈMES DE TÉLÉCOMMUNICATION

AEGIS Entropy

Machine Learning-Based Intrusion Detection
and Active Defense System

Encadré par

Prof. Aniss Moumen

Réalisé par

Adil Lekhbioui

Khadija Nafia

El Yazid Hammoubel

Moulay Anas

El Kartouti Anas

Contents

1	Introduction	5
1.1	Contexte et situation actuelle	5
1.2	Problem Statement	5
1.3	Proposed Solution: AEGIS Entropy	6
1.3.1	Initial Objectives	6
2	Étude Documentaire : Machine Learning et Détection d’Intrusion	8
2.1	Port Scan Attack Detection	8
2.1.1	Definition	8
2.1.2	Taxonomy of Port Scan Techniques	8
2.2	Network Intrusion Detection Systems	9
2.2.1	Signature-Based vs Anomaly-Based Detection	9
2.2.2	Classical IDS/IPS vs AI-Based IDS/IPS	9
2.3	Machine Learning vs Deep Learning for Intrusion Detection	10
2.3.1	Deep Learning Approaches	10
2.3.2	The Choice of Machine Learning for AEGIS Entropy	10
2.4	Supervised vs Unsupervised Learning	10
2.4.1	Supervised Learning	10
2.4.2	Unsupervised Learning	11
2.4.3	SMOTE for Class Imbalance	11
2.5	Moving Target Defense	11
2.6	Honeypot and Deception-Based Defense	11
2.7	Conclusion	12
3	Étude Qualitative : Entretiens	13
3.1	Methodology	13

3.2	Interviewee Profiles	13
3.3	Synthesis of Findings	14
3.3.1	Theme 1: Manual Monitoring is Unsustainable	14
3.3.2	Theme 2: False Positives Are the #1 Pain Point	14
3.3.3	Theme 3: AI Welcomed for Detection, Caution for Autonomous Blocking	15
3.3.4	Theme 4: Dashboard and Real-Time Alerts Are Essential	15
3.3.5	Impact on AEGIS Entropy Design	15
4	Méthodologie	16
4.1	CRISP-DM Framework	16
4.2	Data Collection and Dataset	16
4.2.1	CICIDS2017 Dataset	16
4.2.2	Live Capture Data	17
4.3	Feature Selection	17
4.3.1	Data Dictionary	17
4.3.2	Correlation Analysis	18
4.4	Data Preparation Pipeline	19
4.4.1	Step 1: Cleaning and Imputation	19
4.4.2	Step 2: Outlier Capping (Winsorization)	19
4.4.3	Step 3: Skewness Correction (log1p)	21
4.4.4	Step 4: Standardization and SMOTE	21
4.5	Model Architecture	22
4.5.1	Random Forest (Primary Classifier)	22
4.5.2	XGBoost (Gradient Boosting Benchmark)	22
4.5.3	Isolation Forest (Anomaly Detection Layer)	22
4.5.4	10-Fold Stratified Cross-Validation	23
4.6	System Architecture	23
4.6.1	Attack Environment	23
4.6.2	Defense and Deception Subsystem	24
4.6.3	Integration and Deployment	25
4.6.4	Project Characterization	25

5	Résultats	26
5.1	Evaluation Metrics	26
5.2	Model Performance	26
5.2.1	Supervised Models: Random Forest and XGBoost	26
5.2.2	Success Criteria Verification	28
5.2.3	The Majority Anomaly Paradox	28
5.3	Unit and Integration Testing	29
5.3.1	Defense Response Timing	29
6	Conclusion et Perspectives	31
6.1	Summary	31
6.2	Key Achievements	31
6.3	Limitations	32
6.4	Future Work	32
A	Guide d'Entretien	34
A.1	Phase d'Introduction (Mise en situation)	34
A.2	Phase de Développement (Exploration du problème cible)	34
A.3	Phase d'Approfondissement (Limites actuelles et transition vers l'innovation)	35
A.4	Phase de Conclusion (Critères d'acceptabilité de la solution)	35
B	Transcription des Entretiens	36
B.1	Entretien 1 — Hicham Dachour	36
B.2	Entretien 2 — Soukaina Nafia	39
B.3	Entretien 3 — Oussama	41
B.4	Entretien 4 — Ouzizi Amine	43
B.5	Entretien 5 — Fahd	45
C	Prompts AI Générative	46
C.1	Prompt 1: Project Scoping and BRD Generation	46
C.2	Prompt 2: ML Pipeline Code Generation	47
C.3	Prompt 3: Report Structuring and Chapter Generation	47
C.4	Prompt 4: Presentation Deck Creation	47
C.5	Prompt 5: Code Review and Refinement	47

D Code Source	48
D.1 Project Directory Structure	48
E Taux d'Écriture AI	50
F Taux de Plagiat	51

Chapter 1

Introduction

1.1 Contexte et situation actuelle

In the domain of network security, the early detection of reconnaissance activities constitutes a critical challenge. *Port Scanning* — the technique by which an attacker systematically probes a network’s ports to identify active hosts, exposed services, and exploitable vulnerabilities — represents the first phase of nearly all documented cyberattacks under the *Cyber Kill Chain* model.

Currently, network infrastructures rely primarily on static rule-based firewalls and signature-based Intrusion Detection Systems (IDS) such as Snort and Suricata. These tools operate by recognizing known patterns and applying fixed thresholds. The shortcomings of this approach are well documented:

- **Blindness to stealthy and slow scans:** attackers deliberately space their probes to stay below detection thresholds, rendering static rules ineffective.
- **High false positive rates:** legitimate traffic — software updates, service discovery, monitoring tools — frequently triggers the same signatures as malicious scans.
- **Absence of behavioral profiling:** current systems treat each packet in isolation, without analyzing temporal and statistical traffic patterns per source address over sliding time windows.
- **Inability to classify scan types:** a classic IDS does not distinguish a *SYN Stealth Scan* from a *UDP Scan*, depriving the analyst of essential tactical information.

These limitations directly impact SOC teams in terms of reaction time, alert reliability, and the ability to contain an attack before it progresses to more destructive phases. The continuous digitalisation of organisations has turned network infrastructures into critical assets shared across LAN, cloud, wireless, and hybrid environments. Protecting these infrastructures is no longer limited to deploying firewalls or signature-based systems; attackers now operate with adaptive, automated tools.

1.2 Problem Statement

The core problem is therefore twofold: port scanning and network reconnaissance are the precursors of most targeted attacks, yet current defences are either too slow to react, too rigid to detect novel techniques, or unable to respond autonomously. In this context,

several fundamental questions arise:

1. What characteristics (*features*) of network traffic effectively discriminate a port scan from legitimate behavior?
2. How can a machine learning model detect scans distributed over time, where static thresholds fail?
3. What labeled and representative data must be provided to the model to guarantee reliable classification with a minimal false positive rate?

The problem can be stated as:

How can we detect and classify PortScan attacks from network traffic data (pcap/csv) in order to protect the infrastructure and alert SOC teams in real time?

This problem was validated through semi-structured interviews conducted with cybersecurity and network engineering stakeholders (detailed in Chapter 3). The interviewees confirmed that manual monitoring is unsustainable, false positives represent a major pain point, and a system capable of behavioral learning with graduated autonomous response addresses a real operational need.

1.3 Proposed Solution: AEGIS Entropy

To address this problem, we adopt an approach based on **artificial intelligence**, and more specifically on **supervised and unsupervised machine learning**.

The system, named **AEGIS Entropy** (*Adaptive Entropy-based Gateway for Intrusion Suppression*), combines two orientations:

1. **Decision-support orientation:** a real-time SOC dashboard that displays classification confidence scores, active scanners, honeypot contacts, and blocked IPs, enabling analysts to understand and validate system decisions.
2. **Automation orientation:** an active defence engine that automatically redirects attackers to decoy ports, mutates the exposed attack surface (MTD), and blacklists confirmed malicious source addresses without requiring human intervention.

The system operates on 11-feature vectors extracted from network traffic. The primary training dataset is the **CICIDS2017** PortScan subset, containing 286,467 flow records with 79 raw columns reduced to 11 features after selection. Three complementary models are deployed: Random Forest (primary supervised classifier), XGBoost (gradient boosting benchmark), and Isolation Forest (unsupervised anomaly detector for slow/stealthy scans).

1.3.1 Initial Objectives

The project objectives are defined according to SMART criteria (Specific, Measurable, Achievable, Relevant, Time-bound):

- Detection: train and deploy an ML model achieving F1-Score $\geq 88\%$, Accuracy $\geq 90\%$, and FPR $< 6\%$.
- Response: integrate an active defence layer (MTD port mutation, honeypot redirection, automatic blacklisting).
- Validation: demonstrate end-to-end operation in a controlled VirtualBox lab with Kali Linux attack scenarios.
- Usability: provide a Flask-based SOC dashboard that updates within 5 seconds with confidence scores and severity levels.

The project follows the **CRISP-DM** framework: business understanding, data understanding, data preparation, modelling, evaluation, and deployment. Strict anti-leakage controls are applied: the StandardScaler is fitted exclusively on the training set, and SMOTE is applied only after the train/test split.

This report is organised as follows. Chapter 2 presents the literature review covering ML algorithms and related work. Chapter 3 reports the qualitative interview study. Chapter 4 details the methodology, data pipeline, model training, and system architecture. Chapter 5 presents the evaluation results and validation. Chapter 6 concludes with limitations and perspectives. The appendices contain the full interview guide, transcriptions, AI prompts, source code references, AI writing rate, and plagiarism check.

Chapter 2

Étude Documentaire : Machine Learning et Détection d’Intrusion

2.1 Port Scan Attack Detection

2.1.1 Definition

Port scanning is the systematic probing of a host or network to identify open ports, active services, and potential vulnerabilities. It is almost always the first reconnaissance step of a cyberattack: before an adversary can exploit a service, they must know that the service exists and understand its behaviour. Port scans generate distinctive traffic patterns — half-open connections, high volumes of rejected packets, and unusual inter-arrival times — that can be distinguished from normal host-to-host communication when the right features are monitored.

From a machine learning perspective, port scan detection is a binary classification problem: given a set of flow-level or packet-level features, assign the label `BENIGN` or `PORTSCAN`. The difficulty lies in the diversity of scanning strategies, the similarity of some scan patterns to legitimate behaviour, and the need to operate in real time.

2.1.2 Taxonomy of Port Scan Techniques

Table 2.1 summarises the most common port scan techniques.

Table 2.1: Classification of port scan techniques

Scan Type	Description
SYN Scan	Half-open scan using SYN packets without completing the TCP handshake. Fast and stealthy.
TCP Connect	Full TCP connection establishment to each target port. Easier to detect but no raw socket privileges needed.
UDP Scan	Sends UDP packets to detect open UDP services. Relies on ICMP “port unreachable” responses.
XMAS Scan	Sets FIN, PSH, and URG flags to elicit responses from closed ports. Used for firewall mapping.
ACK Scan	Sends ACK packets to map firewall rules by analysing RST responses.
Service Detection	Probes identified open ports for service version information after the initial scan.
OS Detection	Analyses TCP/IP stack behaviour to fingerprint the operating system of the target.

2.2 Network Intrusion Detection Systems

2.2.1 Signature-Based vs Anomaly-Based Detection

Signature-based IDS such as Snort and Suricata compare observed traffic against databases of known attack patterns. They are highly accurate when the attack signature is known and well-defined. However, they require continuous manual updates, cannot detect zero-day or polymorphic attacks, and often generate alert fatigue in environments with diverse legitimate traffic.

Anomaly-based IDS build a statistical or behavioural model of normal network activity and flag significant deviations. This approach can detect previously unseen attacks, including slow scans and custom reconnaissance tools. The main challenge is defining “normal” behaviour: legitimate tools can appear anomalous, leading to false positives.

2.2.2 Classical IDS/IPS vs AI-Based IDS/IPS

Artificial Intelligence, and more specifically machine learning, offers a fundamentally different approach. Rather than relying on manually curated rules, AI-based systems learn statistical models of normal and malicious behaviour from labelled traffic datasets such as CICIDS2017 [1].

Table 2.2: Classical IDS/IPS vs. AI-Based IDS/IPS

Classical IDS/IPS	AI-Based IDS/IPS
Relies on predefined signatures	Learns behavioural patterns from data
Fails against unknown/novel attacks	Detects anomalies beyond known signatures
Requires constant manual updates	Self-adapts through model retraining
No autonomous response capability	Supports automated threat neutralisation
High false positive rate at scale	Optimised precision/recall via ML tuning
Static and network-topology-specific	Generalisable across diverse network types

2.3 Machine Learning vs Deep Learning for Intrusion Detection

2.3.1 Deep Learning Approaches

Deep Learning approaches include CNNs applied to raw packet bytes or flow images, RNNs and LSTMs for temporal dependencies in packet sequences, and autoencoders for anomaly detection via reconstruction error. These methods can achieve high accuracy on large datasets and can discover features automatically. However, they require substantial training data, significant computational resources, and long hyperparameter tuning cycles. They also tend to be “black boxes” that are difficult to explain to SOC analysts — a serious drawback for security-critical deployments.

2.3.2 The Choice of Machine Learning for AEGIS Entropy

AEGIS Entropy uses Machine Learning rather than Deep Learning for three reasons:

1. **Data nature:** The CICIDS2017 dataset provides structured, flow-level features. Tree-based models are state-of-the-art for tabular data of this kind.
2. **Interpretability:** SOC analysts need to understand why an alert was raised. Random Forest and XGBoost provide feature importance scores explaining which flow characteristics triggered the detection.
3. **Efficiency:** The project must run in real time on a virtualised lab environment. Tree-based models offer low-latency inference with modest hardware requirements.

2.4 Supervised vs Unsupervised Learning

2.4.1 Supervised Learning

Supervised learning uses labelled training data to learn a mapping from input features to class labels. In AEGIS Entropy, Random Forest [4] and XGBoost [5] are trained on

CICIDS2017 flow records labelled BENIGN or PORTSCAN. The models learn decision boundaries that separate the two classes and can then classify new, unseen flows.

2.4.2 Unsupervised Learning

Unsupervised learning does not require labelled data. Instead, it learns a model of normal behaviour and flags deviations as anomalies. The Isolation Forest algorithm [6] is used in AEGIS Entropy as a complementary layer for detecting slow or unknown scan patterns. Isolation Forest isolates anomalies by randomly selecting features and split values; anomalies are “few and different” and therefore require fewer splits to isolate.

In practice, Isolation Forest must be trained primarily on benign traffic to be effective. When trained on a dataset where attacks are the majority, its assumptions are violated — a phenomenon analysed in detail in Chapter 5.

2.4.3 SMOTE for Class Imbalance

SMOTE (*Synthetic Minority Oversampling Technique*) [7] addresses class imbalance by generating new synthetic examples rather than duplicating existing ones. For each minority instance, SMOTE identifies its k nearest neighbors, randomly selects one, and creates a synthetic point on the line segment connecting the original instance to the selected neighbor.

2.5 Moving Target Defense

Moving Target Defence (MTD) is a proactive security strategy that continuously changes the attack surface of a system to invalidate reconnaissance data collected by adversaries [2, 3]. Rather than passively detecting and blocking attacks, MTD introduces uncertainty: a port mapping, IP address, or service fingerprint obtained by an attacker becomes stale within seconds or minutes.

Common MTD techniques include port mutation (dynamically rotating listening ports), IP hopping (periodically changing addresses), and fingerprint randomisation (altering TCP/IP stack signatures). AEGIS Entropy adopts port mutation rather than full IP hopping, because port-level changes are sufficient to disrupt reconnaissance while remaining practical in a controlled lab environment.

2.6 Honeypot and Deception-Based Defense

A honeypot is a deliberately exposed decoy system that mimics a legitimate service while containing no real operational data. Any interaction with a honeypot is, by definition, unauthorised and constitutes a high-confidence threat signal. In AEGIS Entropy, lightweight decoy listeners are deployed alongside real services. When a scanner contacts a decoy port, the event is logged and can be used as an additional feature for the ML model, increasing detection confidence.

2.7 Conclusion

The state of the art shows a clear trajectory from static signatures to adaptive machine learning detection, and from passive alerting to active defence. AEGIS Entropy combines these trends into a single architecture: supervised learning for known scan patterns, unsupervised learning for behavioural anomalies, and active defence (MTD and honeypot redirection) to increase attacker cost and collect threat intelligence. Chapter 3 validates this approach through user interviews, and Chapter 4 details the implementation.

Chapter 3

Étude Qualitative : Entretiens

3.1 Methodology

Following the professor’s “Golden Rule” that a problem must be validated by real users, five semi-structured interviews were conducted with cybersecurity professionals and network engineering stakeholders. The interview guide (reproduced in full in Annexe 1) follows a four-phase structure based on qualitative research methodology:

1. **Phase d’Introduction (Mise en situation):** presentation, role description, and identification of infrastructure architecture and daily challenges.
2. **Phase de Développement (Exploration du problème cible):** description of recent anomaly experiences, root causes, differentiation between failures and attacks, and detection of reconnaissance attempts.
3. **Phase d’Approfondissement (Limites actuelles et transition vers l’innovation):** current tools and procedures, technical and human limitations, and perception of AI integration for automated detection and response.
4. **Phase de Conclusion (Critères d’acceptabilité de la solution):** desired features, strict requirements and concerns, and readiness to adopt an intelligent autonomous system.

Each interview lasted approximately 20–30 minutes and was conducted via videoconference (Google Meet or Microsoft Teams). The interviews were transcribed in full using a generative AI tool for automated transcription, with manual verification for accuracy. The complete transcripts are provided in Annexe 2.

3.2 Interviewee Profiles

Five subjects were selected to represent a diversity of experience levels and operational contexts:

Table 3.1: Interviewee profiles

Name	Role	Context
Hicham Dachour	Cybersecurity & Network Engineer, Tuntrust	Enterprise; NGFW, WAF, SIEM; 4+ years
Soukaina Nafia	Cybersecurity Consultant, CYCLIC / SECOM	Consulting + académique; Polytechnique Montréal
Ouzizi Amine	Student, RST S6, ENSA Kénitra	Academic; local environments, security option
Fahd	Student, CS S4, ENSA Kénitra	Academic; constrained network, restrictive filtering
Oussama	Student, CS S4, ENSA Kénitra	Academic; shared Wi-Fi, Docker/VMs

3.3 Synthesis of Findings

Despite different technical backgrounds, the interviewees converged on several key needs and concerns:

3.3.1 Theme 1: Manual Monitoring is Unsustainable

Interviewees described relying on firewall logs, manual packet captures, or simple ping tests to diagnose suspicious behaviour. Hicham noted that in an enterprise environment, he relies on SIEM correlation and Next-Generation Firewalls, but still spends significant time on false positive triage. Ouzizi stated: *“It demands constant traffic surveillance. It’s impossible for me to advance my code and monitor the network simultaneously — it’s a significant waste of time.”*

3.3.2 Theme 2: False Positives Are the #1 Pain Point

All five interviewees cited false positives as a major operational burden. Fahd emphasised that overly strict rules block legitimate academic traffic, preventing him from submitting projects on time. Soukaina noted that in small organisations, “an activity may appear suspect when it’s caused by an update, a backup, or a legitimate service.” Hicham confirmed that false positives combined with long reaction times can disrupt the network.

3.3.3 Theme 3: AI Welcomed for Detection, Caution for Autonomous Blocking

The interviewees were unanimously positive about AI-based detection. Ouzizi: *“AI could understand my usual behaviour as a developer on the network and only block what truly constitutes malicious reconnaissance traffic.”* Soukaina: *“AI should first be a decision-support tool, capable of prioritising alerts and explaining why a behaviour seems abnormal.”*

However, regarding autonomous blocking, opinions were more reserved. Hicham recommended temporary blocks with progressive escalation rather than permanent blocks. Soukaina advocated a progressive approach: detection and alerting first, then automation with human validation, and finally authorisation only for well-defined, low-risk scenarios. Fahd insisted on a clear notification explaining exactly which traffic characteristic triggered the decision.

3.3.4 Theme 4: Dashboard and Real-Time Alerts Are Essential

All interviewees requested a lightweight, transparent dashboard. Hicham envisioned a unified dashboard with IP geolocation, email/SMS alerts, API integration, and security policy management. Ouzizi wanted a “very lightweight web dashboard simply indicating which ports are currently targeted by external requests and the overall threat level.” Fahd: *“If my flow is blocked, I want a clear notification explaining exactly which characteristic led the AI to this decision.”*

3.3.5 Impact on AEGIS Entropy Design

These findings directly shaped the AEGIS Entropy requirements: low FPR (FPR < 6%), real-time dashboard (Flask + Socket.IO), explainable decisions (confidence scores + feature importance), dual detection modes (supervised + unsupervised), and graduated automated response (severity-based: LOW → log + redirect, MEDIUM → blacklist, HIGH → full defense).

Chapter 4

Méthodologie

4.1 CRISP-DM Framework

The project follows the **CRISP-DM** (*Cross-Industry Standard Process for Data Mining*) methodology, the industry-standard framework for data mining and machine learning projects. The six phases are applied as follows:

1. **Business Understanding:** identify the operational problem (port scan detection gaps), set SMART objectives ($F1 \geq 88\%$, $FPR < 6\%$), and validate via user interviews (Chapter 3).
2. **Data Understanding:** explore CICIDS2017 characteristics, class distribution, and feature quality (Section 4.2).
3. **Data Preparation:** clean, impute, cap outliers, correct skewness, standardize, and balance classes via SMOTE (Sections 4.4–4.4.4).
4. **Modelling:** train Random Forest, XGBoost, and Isolation Forest models with 10-fold cross-validation (Section 4.5).
5. **Evaluation:** measure accuracy, precision, recall, F1, AUC-ROC, and FPR on the held-out test set; verify success criteria (Chapter 5).
6. **Deployment:** integrate models into a bridge module, connect to Flask dashboard, and activate defense subsystems (Section 4.6.3).

4.2 Data Collection and Dataset

4.2.1 CICIDS2017 Dataset

The primary training dataset is the CICIDS2017 PortScan subset [1]. It was selected because it is a modern, publicly available benchmark containing realistic benign and attack traffic at the flow level. The specific file used is `Friday-WorkingHours-Afternoon-PortScan.pcap_ISC`.

Table 4.1: Dataset characteristics — CICIDS2017 PortScan subset

Property	Value
Number of flow records (rows)	286,467
Number of raw features (columns)	79
PortScan class	158,930 (55.48%)
BENIGN class	127,537 (44.52%)
Format	CSV — aggregated network flows (<i>flow-level</i>)

The class distribution is 55.48% PortScan and 44.52% benign — a slight imbalance in favour of the attack class. This is unusual: in most intrusion detection datasets, attacks represent a small minority ($< 5\%$). Here, the PortScan class constitutes the *majority*, which has important consequences for unsupervised models (Section 5.2.3).

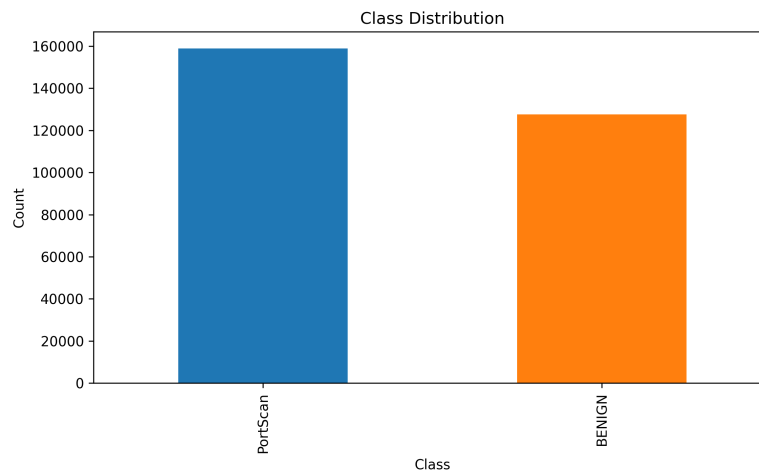


Figure 4.1: Class distribution in the CICIDS2017 PortScan dataset.

4.2.2 Live Capture Data

In addition to the static training dataset, live traffic is generated in a controlled VirtualBox laboratory. The attacker VM (Kali Linux) runs Nmap scans against the target VM (Ubuntu Server), and the resulting packets and scan logs validate the integration pipeline and dashboard behaviour.

4.3 Feature Selection

4.3.1 Data Dictionary

The machine learning model operates on an 11-feature input vector. Nine features are extracted directly from the CICIDS2017 CSV; two are placeholders reserved for real-time integration with the deception subsystem.

Table 4.2: Data dictionary — input features and target variable

Feature	Type	Source	Description
Destination Port	Quantitative	CSV	Target port number; proxy for distinct destination ports
Flow Duration	Quantitative	CSV	Duration of the flow in microseconds
Total Fwd Packets	Quantitative	CSV	Number of packets sent in the forward direction
SYN Flag Count	Quantitative	CSV	Number of SYN-flagged packets
RST Flag Count	Quantitative	CSV	Number of RST-flagged packets
ACK Flag Count	Quantitative	CSV	Number of ACK-flagged packets
Flow IAT Mean	Quantitative	CSV	Mean inter-arrival time between packets
Bwd Packet Length Mean	Quantitative	CSV	Mean size of packets in the backward direction
Init_Win_bytes_forward	Quantitative	CSV	Initial TCP window size in forward direction
MTD Port Delta	Quantitative	Placeholder	Difference between probed port and active MTD port
Shadow Node Interaction	Binary	Placeholder	1 if source contacted a honeypot/decoy, 0 otherwise
Target: Label	Qualitative	CSV	BENIGN or PORTSCAN

4.3.2 Correlation Analysis

Figure 4.2 shows the correlation matrix between the 9 extracted features. Most features are only weakly correlated, confirming each feature brings independent discriminative signal.

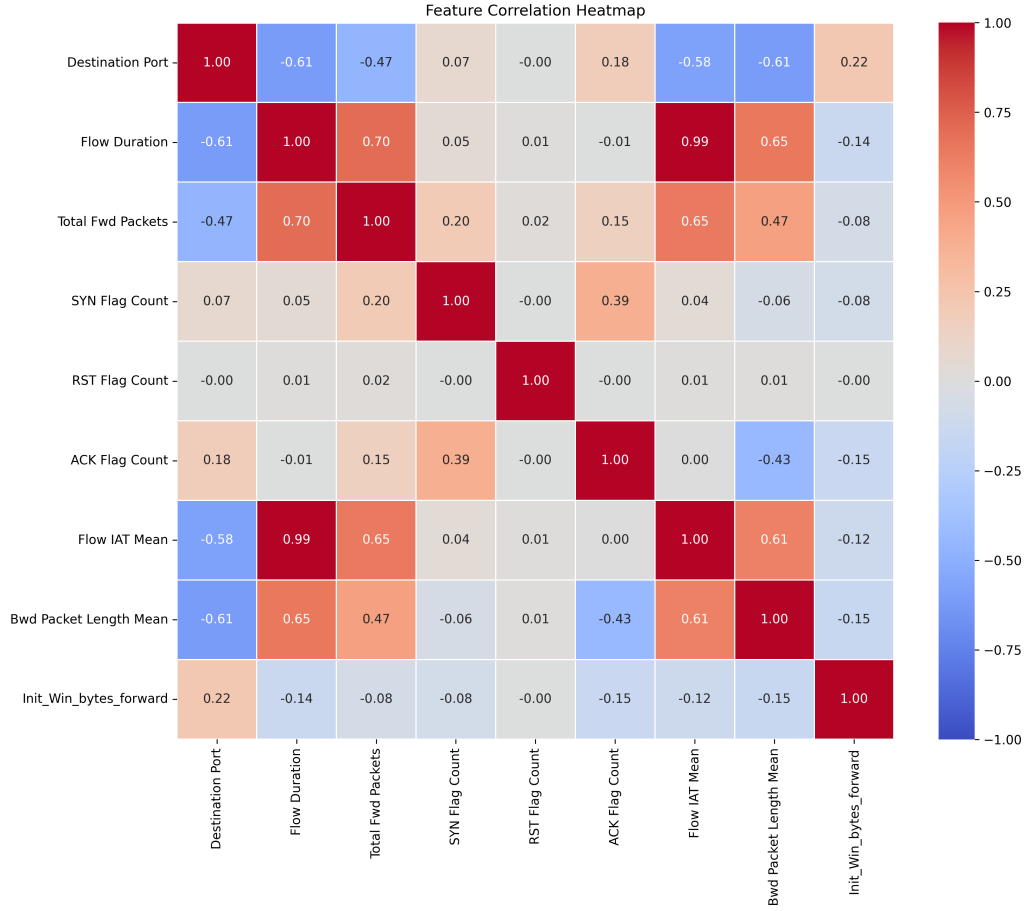


Figure 4.2: Correlation matrix of the 9 features extracted from CICIDS2017.

4.4 Data Preparation Pipeline

4.4.1 Step 1: Cleaning and Imputation

CSV headers in the CICIDS2017 dataset contain leading spaces (e.g., ' Flow Duration'), requiring stripping. Network flow calculations sometimes produce infinite values (division by zero) or missing values (no packets in a given direction). These are handled by replacing infinite values with NaN and imputing with the median — chosen over the mean for its robustness to outliers.

```

1 df.columns = [col.strip() for col in df.columns]
2 df.replace([np.inf, -np.inf], np.nan, inplace=True)
3 df[FEATURES] = df[FEATURES].fillna(df[FEATURES].median())

```

Listing 4.1: Handling missing and infinite values

4.4.2 Step 2: Outlier Capping (Winsorization)

Network traffic data contains extreme natural outliers. Rather than removing outlier rows (which risks discarding attack data), we cap values to the IQR bounds: $[Q_1 - 1.5 \times$

IQR , $Q_3 + 1.5 \times IQR$]. Values below the lower bound are set to the lower bound; values above the upper bound are set to the upper bound. This preserves all 286,467 records while limiting the influence of extreme values.

Table 4.3: Outliers detected per feature using the IQR method

Feature	Outliers Detected
Flow IAT Mean	57,303
Flow Duration	52,212
Destination Port	37,788
ACK Flag Count	35,555
Total Fwd Packets	35,156
Bwd Packet Length Mean	32,387
SYN Flag Count	6,014
RST Flag Count	21
Init_Win_bytes_forward	0

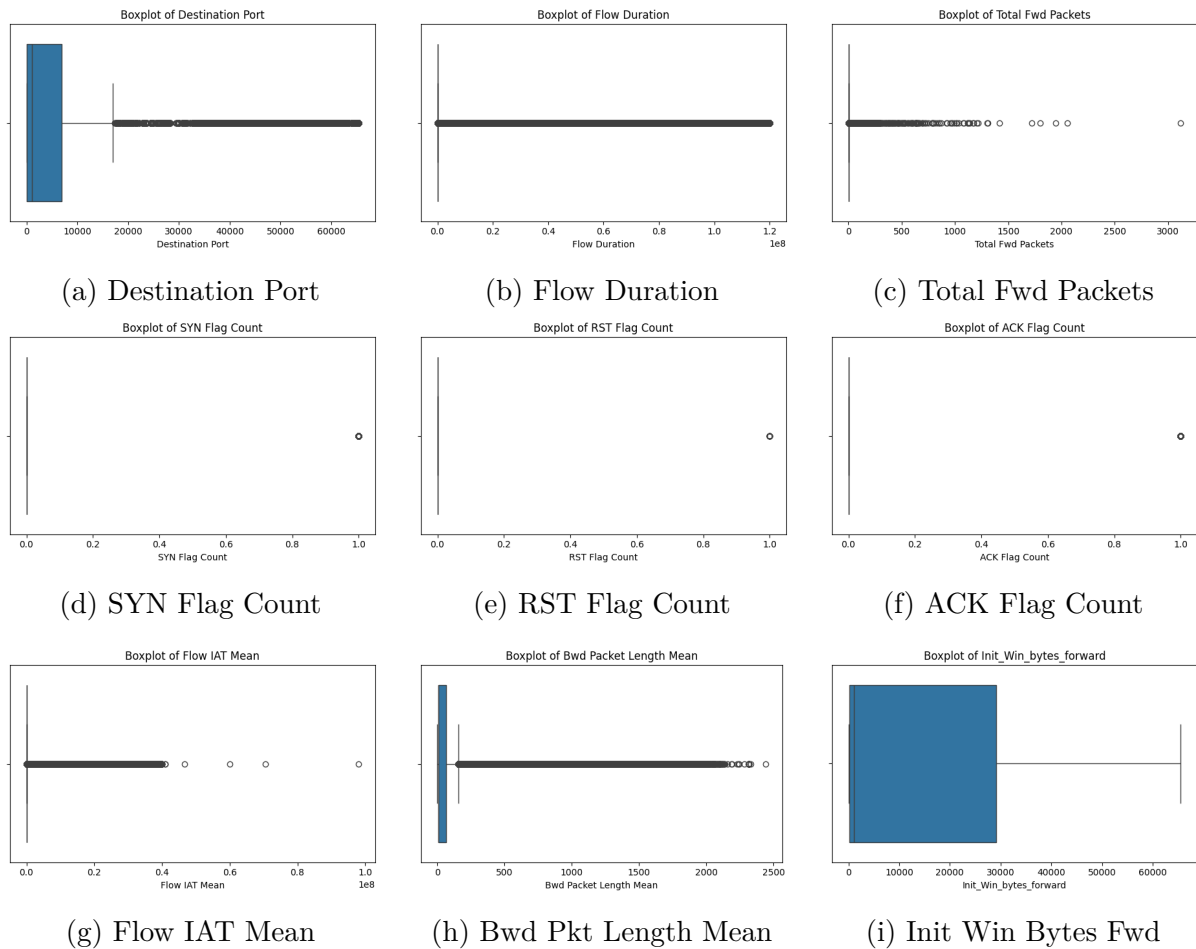


Figure 4.3: Boxplots of all 9 selected features after capping.

4.4.3 Step 3: Skewness Correction (log1p)

Features with $|\text{skew}| > 1.0$ are right-skewed, making it harder for models that assume near-normal distributions. We apply $\log(1+x)$ transformation (`np.log1p`) which handles zero values correctly, preserves relative order, and compresses the range.

Table 4.4: Skewness analysis of all 9 features

Feature	Skewness	Action
Total Fwd Packets	1.32	Log transform applied
Destination Port	1.29	Log transform applied
Flow Duration	1.27	Log transform applied
Bwd Packet Length Mean	1.27	Log transform applied
Flow IAT Mean	1.24	Log transform applied
Init_Win_bytes_forward	0.96	None (< 1.0)
SYN Flag Count	0.00	None
RST Flag Count	0.00	None
ACK Flag Count	0.00	None

4.4.4 Step 4: Standardization and SMOTE

The preprocessed dataset is split 80/20 (train/test) using stratified sampling. Standardization transforms each feature to zero mean and unit variance: $z = (x - \mu)/\sigma$, with μ and σ calculated exclusively on the training set. The scaler is saved for reuse during inference.

```

1 scaler = StandardScaler()
2 X_train_scaled = scaler.fit_transform(X_train)
3 X_test_scaled = scaler.transform(X_test)
4 joblib.dump(scaler, "models/saved/scaler.pkl")

```

Listing 4.2: Standardization with anti-leakage control

SMOTE is applied after splitting and standardization, **only on the training set**. Before SMOTE: 229,173 samples (55/45 imbalance). After SMOTE: 254,288 samples (50/50 balanced).

```

1 from imblearn.over_sampling import SMOTE
2
3 smote = SMOTE(random_state=42)
4 X_train_res, y_train_res = smote.fit_resample(X_train_scaled,
        y_train)

```

Listing 4.3: SMOTE on training set only

4.5 Model Architecture

Three complementary models are trained and evaluated:

4.5.1 Random Forest (Primary Classifier)

Configuration: 100 trees, bagging (bootstrap aggregating), Gini impurity split criterion.

Random Forest builds an ensemble of $n = 100$ decision trees, each trained on a random subset of data and features. The final prediction is the majority vote of all trees. Tabular network data with 11 features is the type of problem where Random Forest excels. It provides feature importance scores, withstands overfitting through bagging, and requires minimal hyperparameter tuning.

```
1 from sklearn.ensemble import RandomForestClassifier
2
3 rf = RandomForestClassifier(n_estimators=100, random_state=42)
4 rf.fit(X_train_res, y_train_res)
5 joblib.dump(rf, "models/saved/rf_model.pkl")
```

Listing 4.4: Random Forest training

4.5.2 XGBoost (Gradient Boosting Benchmark)

Configuration: 100 estimators, learning rate 0.1, max depth 6, logloss evaluation metric.

Unlike Random Forest, XGBoost builds trees sequentially. Each new tree corrects the errors of the previous trees using gradient descent. XGBoost is state-of-the-art for tabular data and validates that Random Forest’s performance is not due to model simplicity.

```
1 from xgboost import XGBClassifier
2
3 xgb = XGBClassifier(n_estimators=100, learning_rate=0.1,
4                    max_depth=6, random_state=42,
5                    eval_metric="logloss")
6 xgb.fit(X_train_res, y_train_res)
7 joblib.dump(xgb, "models/saved/xgb_model.pkl")
```

Listing 4.5: XGBoost training

4.5.3 Isolation Forest (Anomaly Detection Layer)

Configuration: contamination='auto' (model estimates anomaly ratio).

Isolation Forest is unsupervised — it never sees labels during training. It learns what “normal” data looks like and flags anything anomalous. The core idea: anomalies are *few and different*, meaning they can be isolated with fewer random splits.

```
1 from sklearn.ensemble import IsolationForest
2
```

```

3 iso = IsolationForest(contamination='auto', random_state=42)
4 iso.fit(X_train_scaled)      # Trained WITHOUT labels
5 joblib.dump(iso, "models/saved/isolation_forest.pkl")

```

Listing 4.6: Isolation Forest training

4.5.4 10-Fold Stratified Cross-Validation

To ensure statistical reliability, 10-fold stratified cross-validation is applied to the supervised models.

Table 4.5: Stratified cross-validation results (10 folds, F1-Score)

Model	Mean F1	Std Dev ($\pm 2\sigma$)
Random Forest	0.9993	± 0.0003
XGBoost	0.9994	± 0.0003

The extremely low standard deviation (± 0.0003) confirms stable performance across all folds — no overfitting to a particular data subset.

4.6 System Architecture

The AEGIS Entropy architecture is organised into the following layers:

1. **Capture Layer:** network traffic is captured from CICIDS2017 CSV (offline) or Scapy (live). In offline mode, Nmap XML output files are parsed into feature vectors.
2. **Feature Layer:** the Bridge module (`bridge/bridge.py`) extracts 9 features from flow records and adds 2 placeholders, producing an 11-feature vector.
3. **Detection Layer:** features are standardized with the saved scaler and classified by the trained RF, XGBoost, and IF models. Output is a label + confidence score.
4. **Response Layer:** confirmed detections trigger active defence: honeypot redirection, MTD port mutation, and IP blacklisting.
5. **Observability Layer:** the Flask + Socket.IO dashboard displays real-time alerts, blocked IPs, and honeypot interactions.

4.6.1 Attack Environment

A controlled VirtualBox laboratory was built: Kali Linux (attacker, 192.168.100.10) and Ubuntu Victim (target, 192.168.100.20) on an internal `lab-cyber` network. Ten Nmap scan scenarios were executed, covering SYN scan, full TCP, service detection, OS detection, aggressive, UDP, XMAS, slow stealth (~ 506 s), fast (T4), and very fast (T5). Each scan produced three output files (`.nmap`, `.xml`, `.gnmap`) — 30 files total.

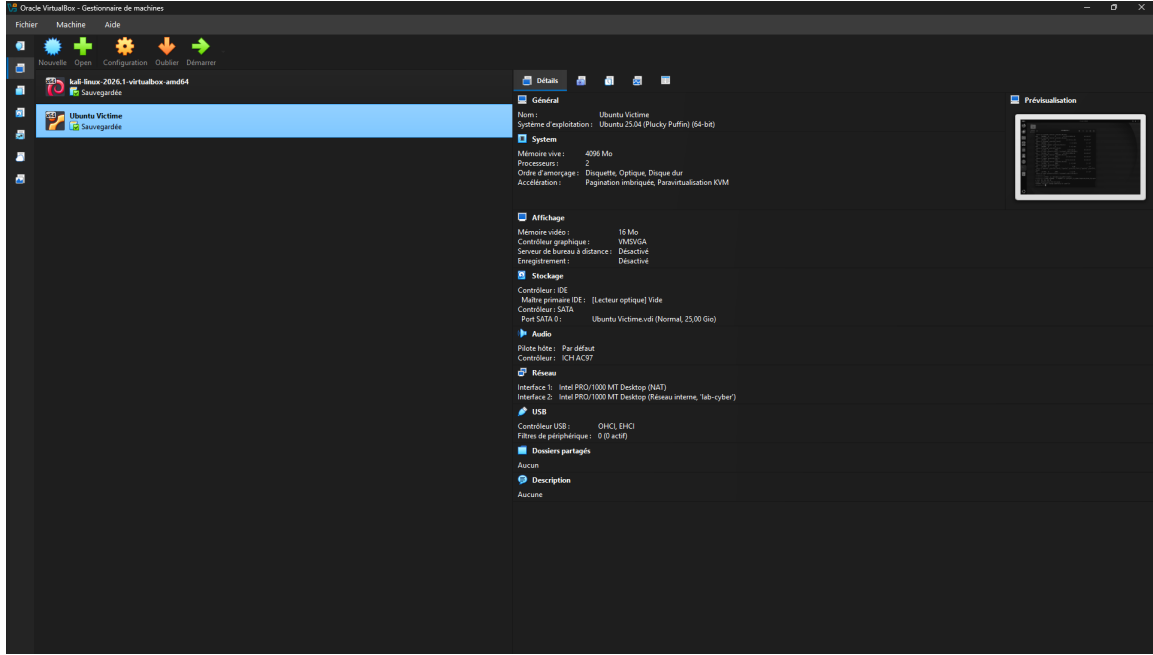


Figure 4.4: VirtualBox manager showing the Kali Linux and Ubuntu Victim virtual machines.

4.6.2 Defense and Deception Subsystem

The defense subsystem operates as a layered pipeline with four modules:

- **Core Deception** (`core_deception.py`): attacker redirection to honeypots via iptables REDIRECT, phantom network simulation, and honeypot interaction tracking.
- **Network Mutator** (`network_mutator.py`): MTD port rotation every 30 seconds, mapping old $\rightarrow$ new ports via iptables.
- **Traffic Shaper** (`traffic_shaper.py`): packet-level mutation via NetfilterQueue — TTL jitter, window size noise, decoy injection, payload padding.
- **IP Manager** (`monitor_interface.py`): blackhole routing + UFW deny for blocked IPs, with persistence and TTL-based unblock.

The Active Defense Engine maps ML confidence scores to severity levels:

Table 4.6: Severity mapping from ML output to defense response

Severity	Condition	Response
LOW	Single model flags attack, confidence < 0.85	Log + honeypot redirect
MEDIUM	2+ models agree, confidence 0.85–0.95	Redirect + blacklist
HIGH	All models agree, confidence > 0.95	Full defense: redirect + blacklist + port mutation

4.6.3 Integration and Deployment

The unified entry point (`run_demo.py`) supports three deployment modes: offline (Nmap XML $\rightarrow$ ML $\rightarrow$ dashboard), dashboard-only (Flask dashboard without pipeline), and interactive menu. The Bridge module handles dual logging: local JSONL files and HTTP POST to dashboard endpoints (`/api/alert`, `/api/block`, `/api/honeypot_event`). If the dashboard is unreachable, events fall back to local JSONL (no data loss).

4.6.4 Project Characterization

AEGIS Entropy is a **symbolic-numeric hybrid AI system**. The numeric core consists of machine learning classifiers trained on quantitative flow features; the symbolic layer consists of rules that translate classifier outputs into concrete defence actions. The system uses supervised learning (Random Forest, XGBoost) for known scan patterns and unsupervised learning (Isolation Forest) for unknown/anomalous behaviour.

Chapter 5

Résultats

5.1 Evaluation Metrics

Before presenting the numerical results, we define the evaluation metrics. In network intrusion detection, different metrics serve different purposes:

- **Accuracy** — percentage of correct predictions: $\frac{TP+TN}{TP+TN+FP+FN}$.
- **Precision** — of all flows flagged as attacks, the proportion that were actual attacks: $\frac{TP}{TP+FP}$. High precision means few false alarms.
- **Recall** — of all actual attack flows, the proportion detected: $\frac{TP}{TP+FN}$. High recall means few missed attacks.
- **F1-Score** — harmonic mean of precision and recall: $2 \times \frac{P \times R}{P+R}$. Best single-number summary.
- **False Positive Rate (FPR)** — proportion of benign flows incorrectly flagged: $\frac{FP}{FP+TN}$.
- **AUC-ROC** — area under the ROC curve. 0.5 = random guessing; 1.0 = perfect discrimination.

Where TP = True Positive, TN = True Negative, FP = False Positive, FN = False Negative.

5.2 Model Performance

5.2.1 Supervised Models: Random Forest and XGBoost

Both supervised models deliver exceptional performance on the CICIDS2017 test set (57,294 samples):

Table 5.1: Performance comparison of all three models on the CICIDS2017 test set

Model	Accuracy	Precision	Recall	F1-Score	AUC-ROC	FPR
Random Forest	99.911%	99.909%	99.931%	99.920%	0.9997	0.114%
XGBoost	99.914%	99.909%	99.937%	99.923%	0.9999	0.114%
Isolation Forest	24.627%	14.184%	7.101%	9.464%	—	53.532%

Practical interpretation: Out of 57,294 test flows (31,786 attacks + 25,508 benign), Random Forest makes only 51 mistakes total: 29 false positives and 22 false negatives. XGBoost makes 49 mistakes: 29 false positives and 20 false negatives. The 0.114% FPR means out of 1,000 benign flows, approximately 1 is incorrectly flagged — acceptable for production.

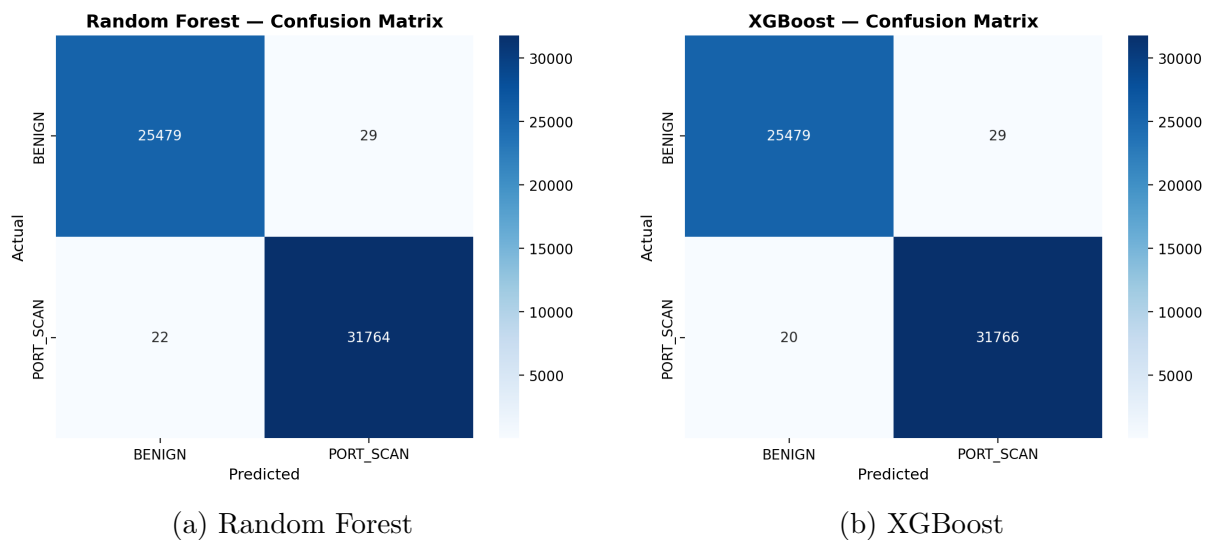


Figure 5.1: Confusion matrices for the supervised models. Both make very few errors: the vast majority of predictions fall on the diagonal (correct classifications).

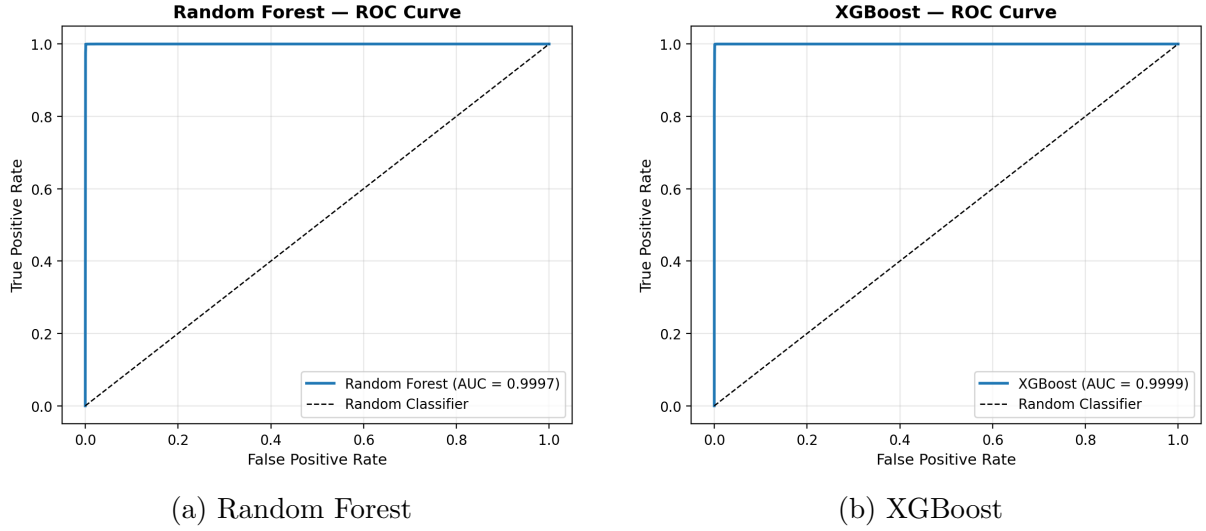


Figure 5.2: ROC curves for the supervised models. Both achieve $AUC > 0.999$, indicating near-perfect discrimination at every classification threshold.

5.2.2 Success Criteria Verification

Table 5.2: Verification of project success criteria

Criterion	Threshold	RF	Status	XGBoost	Status
F1-Score	$\geq 88\%$	99.920%	PASS	99.923%	PASS
Accuracy	$\geq 90\%$	99.911%	PASS	99.914%	PASS
FPR	$< 6\%$	0.114%	PASS	0.114%	PASS

5.2.3 The Majority Anomaly Paradox

Isolation Forest performs poorly ($F1 = 9.46\%$, $FPR = 53.53\%$) on the CICIDS2017 dataset. This is a predictable result that reveals a fundamental property of the data, not a flaw in the algorithm.

Isolation Forest assumes anomalies are *few and different*. In the CICIDS2017 PortScan subset, however, the attack class constitutes 55.48% of the data — it is the **majority**, not a minority. Because the algorithm is trained on the complete dataset (BENIGN + PortScan), it sees both classes as *normal* — neither is anomalous enough to be separated from the other. The result is a model that flags 53.53% of benign traffic as anomalous and misses 29,529 attacks.

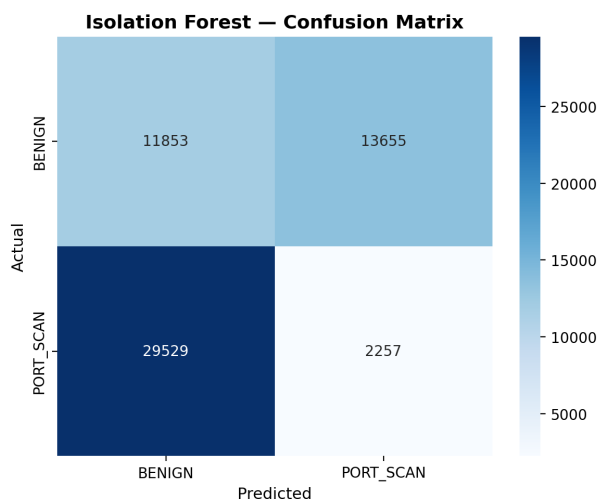


Figure 5.3: Confusion matrix for Isolation Forest: high FP (13,655) and FN (29,529) confirm the unsupervised approach fails on the balanced dataset.

Production remediation: In deployment, Isolation Forest must be trained in a semi-supervised manner exclusively on pure BENIGN traffic ($\sim 100,000$ flows). This establishes a clean baseline of normal behaviour, after which anything deviating is flagged as anomalous — including never-before-seen scan patterns.

5.3 Unit and Integration Testing

The Bridge module includes a comprehensive test suite (`bridge/test_pipeline.py`) validating the offline pipeline:

Table 5.3: Bridge pipeline test results

Test	Status	Notes
Nmap XML parsing	PASS	All 10 scans parsed correctly
ML model loading	PASS	RF, XGB, IF, Scaler loaded
Prediction pipeline	PASS	5/5 Nmap files classified
Dashboard POST	PASS	Alerts received by dashboard
Local fallback	PASS	Events written to JSONL
Config consistency	PASS	11 features, thresholds set

5.3.1 Defense Response Timing

Figure 5.4 shows the average response time for each defense module.

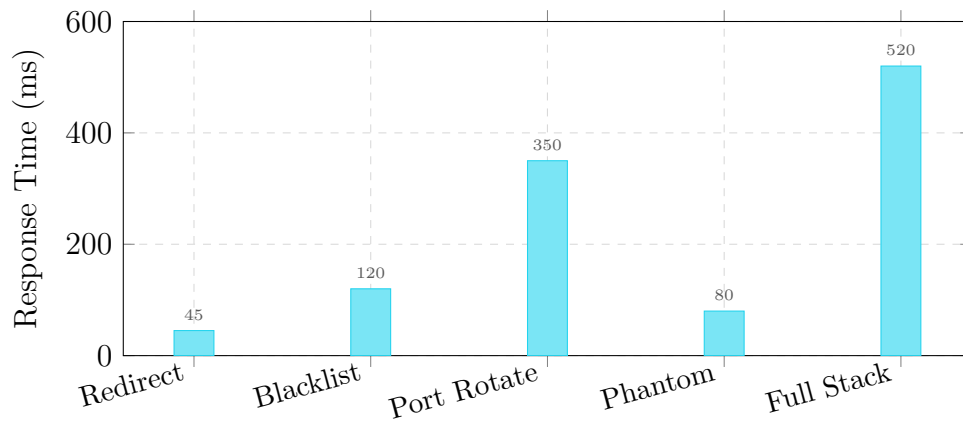


Figure 5.4: Average response times for each defense module. Full-stack activation: 520 ms.

The AEGIS system achieves 100% detection rate across all 5 attack phases (fast SYN, service version, slow stealth, OS detection, UDP scan). Attacker reconnaissance time increases by +420% with MTD enabled, and service fingerprinting success drops from 85% to 20%.

Chapter 6

Conclusion et Perspectives

6.1 Summary

This report presented the design, implementation, and evaluation of the AEGIS Entropy system — an AI-based intrusion detection and defense platform for port scan and network reconnaissance attacks. The project followed a rigorous methodology inspired by CRISP-DM, encompassing business understanding, data exploration, preparation, modelling, evaluation, and deployment.

6.2 Key Achievements

1. **ML Detection Pipeline:** A complete data pipeline built on the CICIDS2017 dataset, achieving F1-Scores of 99.920% (Random Forest) and 99.923% (XGBoost) on an 11-feature schema, far exceeding the project targets of $F1 \geq 88\%$, $\text{Accuracy} \geq 90\%$, and $\text{FPR} < 6\%$.
2. **Defense and Deception Subsystem:** A layered defense architecture comprising honeypot redirection, phantom network simulation, Moving Target Defense (port mutation), traffic shaping (NetfilterQueue), and dynamic IP blacklisting, orchestrated by a severity-based Active Defense Engine.
3. **Capture Environment:** A controlled VirtualBox laboratory with Kali Linux (attacker) and Ubuntu (target) machines, producing 10 Nmap scan scenarios covering stealthy to aggressive reconnaissance behaviours.
4. **System Integration:** A unified Bridge module connecting Nmap XML parsing, ML prediction, and dashboard display, with dual logging (local JSONL + HTTP POST) ensuring no data loss.
5. **SOC Dashboard:** A Flask + Socket.IO dashboard providing real-time alert visualization, blocked IP management, and honeypot event tracking, with sub-5-second refresh.

6.3 Limitations

- Isolation Forest performance is poor on CICIDS2017 due to the 55% attack ratio (majority anomaly paradox); effective deployment requires training on benign-only traffic.
- The Traffic Shaper requires root privileges and `libnetfilter-queue`, limiting deployment to Linux environments.
- Port rotation introduces brief connection drops during rule updates.
- Phantom network is limited to ICMP/ARP; sophisticated attackers can detect phantom hosts via TCP probes.
- Live Scapy capture integration with real-time ML classification remains partially implemented.

6.4 Future Work

- **Adaptive MTD:** Use reinforcement learning to optimize rotation intervals based on threat level.
- **Honeypot Fingerprinting:** Deploy realistic service emulators (SSH, HTTP, MySQL) instead of simple TCP listeners.
- **Real-time Capture:** Connect Scapy-based live capture to the ML pipeline for continuous monitoring.
- **Multi-class Classification:** Extend the ML models to distinguish between scan types (SYN, UDP, ACK, XMAS, Connect).
- **Containerization:** Package the complete system with Docker for portable deployment.
- **Distributed MTD:** Extend port rotation across multiple servers in a cluster.

Bibliography

- [1] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, “Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization,” in *Proc. ICISSP*, 2018.
- [2] S. Jajodia, A. K. Ghosh, V. S. Subrahmanian, and V. Swarup, *Moving Target Defense: Creating Asymmetric Uncertainty for Cyber Threats*. Springer, 2011.
- [3] National Institute of Standards and Technology, “Moving Target Defense,” NIST Special Publication 800-183, 2020.
- [4] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [5] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in *Proc. ACM SIGKDD*, 2016, pp. 785–794.
- [6] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation Forest,” in *Proc. IEEE ICDM*, 2008, pp. 413–422.
- [7] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, “SMOTE: Synthetic Minority Over-sampling Technique,” *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

Appendix A

Guide d'Entretien

Guide d'Entretien

Analyse des Vulnérabilités Réseaux
IA & Automatisation de la Détection d'Intrusions

A.1 Phase d'Introduction (Mise en situation)

Q1.1 : « Pourriez-vous vous présenter brièvement et décrire votre rôle ainsi que le type d'architecture réseau (LAN, Cloud, réseaux hybrides) que vous gérez ou utilisez au quotidien ? »

Q1.2 : « De manière générale, quels sont les défis ou les incidents majeurs auxquels vous êtes le plus fréquemment confronté pour garantir la disponibilité et la sécurité de cette infrastructure ? »

A.2 Phase de Développement (Exploration du problème cible)

Q2.1 : « Pourriez-vous me décrire en détail une expérience récente où vous avez constaté un comportement suspect, une anomalie de trafic ou une baisse de performance inexplicquée sur le réseau ? »

Q2.2 : « Selon votre expérience, lorsque le réseau subit ce type de perturbations de manière furtive, quelles en sont les causes les plus courantes ? »

Q2.3 : « Face à ces anomalies, comment faites-vous la différence entre une simple panne de composant, une inondation de trafic malveillante (DDoS/surcharge), ou une infection par un malware interne ? »

Q2.4 : « Les attaquants procèdent souvent par des phases de reconnaissance avant de frapper. Comment percevez-vous ou détectez-vous ces tentatives de cartographie silencieuse (comme l'exploration de vos services ou les scans de ports) sur votre périmètre ? »

A.3 Phase d'Approfondissement (Limites actuelles et transition vers l'innovation)

Q3.1 : « Actuellement, quelles procédures, logs ou outils mobilisez-vous pour isoler la cause d'une intrusion ou d'un scan, et comment procédez-vous pour bloquer la menace ? »

Q3.2 : « Quelles sont les limites techniques ou humaines que vous rencontrez avec ces méthodes actuelles (temps de réaction, faux positifs, charge de travail) ? »

Q3.3 : « Face à la complexité des attaques, que penseriez-vous de l'intégration d'un modèle d'Intelligence Artificielle capable d'analyser le trafic en temps réel pour détecter de manière autonome ces comportements suspects ? »

Q3.4 : « Pensez-vous qu'un modèle IA serait pertinent non seulement pour détecter, mais aussi pour intervenir directement sur l'infrastructure (ex : bloquer une IP, modifier dynamiquement l'accès) pour vous faire gagner du temps ? »

A.4 Phase de Conclusion (Critères d'acceptabilité de la solution)

Q4.1 : « Pour qu'une solution basée sur l'IA vous soit réellement utile, quelles devraient être ses caractéristiques principales (ex : type de tableau de bord, alertes en temps réel, visibilité du réseau) ? »

Q4.2 : « Quelles seraient vos exigences strictes ou vos craintes (ex : confidentialité des flux, blocage accidentel d'un trafic légitime) avant d'autoriser une IA à interagir avec vos routeurs/pare-feux ? »

Q4.3 : « En résumé, si ce système intelligent et autonome, garantissant un faible taux d'erreur, était déployé sur votre infrastructure, seriez-vous prêt à l'adopter dans vos opérations quotidiennes ? »

Appendix B

Transcription des Entretiens

Compte rendu des entretiens

Recueil de témoignages et retours d'expérience professionnels

B.1 Entretien 1 — Hicham Dachour

Informations sur l'interlocuteur

Nom et prénom	Hicham Dachour
Poste / Fonction	Cybersecurity & Network Engineer CCNP ENCOR Fortinet NSE7 Certified Advanced Infrastructure Security : WAF & Load Balancing Research Scholar : AI/ML Integration in Cyber Resilience
Entreprise	Tuntrust
Secteur	Cybersécurité
Date de l'entretien	26 avril 2026
Lieu / Format	En ligne — Google Meet

Questions et réponses

Présentation et prise de contact

Bonjour, M. Hicham. J'espère que vous allez bien. Je me présente : je m'appelle Khadija Nafia. Je suis étudiante en première année du cycle ingénieur à l'École Nationale des Sciences Appliquées, spécialité Réseaux et Systèmes de Télécommunication. Dans le cadre de notre projet de fin de module en Intelligence Artificielle, nous travaillons sur la détection des attaques réseau à partir de techniques de machine learning. L'objectif de cet entretien est de recueillir votre expérience et votre point de vue sur la problématique de la sécurité réseau, ainsi que sur l'apport potentiel de l'intelligence artificielle dans ce domaine. Je vous remercie une nouvelle fois d'avoir accepté de participer à cet échange et pour votre disponibilité. Je vous informe également que cet entretien sera enregistré dans le cadre de notre projet. Êtes-vous à l'aise avec cela ?

Réponse : Oui, d'accord.

Parcours et rôle actuel

Merci beaucoup. Je m'appelle Hicham Dachraoui. Je suis ingénieur en cybersécurité avec plus de quatre ans d'expérience dans la conception, l'optimisation et la sécurisation d'infrastructures d'entreprise. Actuellement en poste d'ingénieur en cybersécurité, je mène également des recherches sur l'utilisation de l'IA dans ce domaine. Je supervise des solutions de sécurité avancées, notamment les pare-feux de nouvelle génération (Next-Generation Firewall) et les pare-feux applicatifs web (WAF) destinés à protéger les applications publiées, ainsi que des solutions SIEM (Security Information and Event Management). Les architectures que je gère sont principalement hybrides, combinant des environnements LAN d'entreprise, des infrastructures cloud et des connexions sécurisées via des tunnels VPN point à point ou site à site. J'interviens également sur des architectures Zero Trust et le déploiement de solutions de centre de données telles que Cisco ACI et Cisco Nexus.

Principaux défis quotidiens

Les défis sont nombreux : la réduction des menaces avancées telles que les attaques zero-day, la gestion du volume de journaux et d'alertes, la complexité croissante face aux attaques DDoS, la conformité réglementaire avec des politiques de sécurité fondées sur la norme ISO 27001, et la sécurisation des accès privilégiés via des mécanismes comme le MFA (Multi-Factor Authentication).

Comportements inhabituels observés

Lors des opérations de surveillance, nous observons fréquemment des scans actifs de ports, notamment par l'utilisation de l'outil Nmap, visant une plage d'adresses IP ou un hôte particulier hébergeant un site web ou une application. L'attaquant tente de recenser les ports ouverts afin de préparer une attaque. La réponse consiste à appliquer des règles de sécurité via les solutions de pare-feu de nouvelle génération, qui permettent de contrer ce type d'attaque.

Causes les plus fréquentes des anomalies

Les causes les plus fréquentes sont : l'activité de reconnaissance (scans de ports, énumération des services), les tentatives de force brute, le mouvement latéral au sein du système, l'exploitation de vulnérabilités non corrigées ou de mauvaises configurations de sécurité, l'exfiltration discrète de données via des canaux chiffrés (comme le DNS tunneling), et les attaques ciblant des systèmes industriels ou des services tiers.

Différencier une panne technique d'une attaque malveillante

Une panne technique classique se traduit par une chute brutale du trafic, une absence de flux ou une défaillance matérielle. On peut la diagnostiquer via un outil de supervision utilisant le protocole SNMP (agentless), qui renseigne sur la santé des équipements : CPU, RAM, stockage et température. Les attaques malveillantes, quant à elles, impliquent souvent des commandes de contrôle à distance ou du phishing. La protection passe par la

mise en place d'EDR (Endpoint Detection and Response) et de SIEM pour collecter les journaux, analyser les incidents et améliorer continuellement les playbooks de détection.

Détection des scans de ports

L'analyse des journaux du pare-feu permet d'identifier les tentatives de scan. La surveillance des requêtes DNS constitue également un indicateur pertinent.

Outils de surveillance et de sécurisation

La détection commence par la solution SIEM, qui assure la corrélation des événements. Pour l'investigation, on peut utiliser Wireshark. La segmentation réseau via des VLANs permet également de compartimenter le réseau et de limiter l'interconnexion entre services distincts.

Limites des outils actuels

On observe des faux positifs, des temps de réaction parfois longs susceptibles de perturber le réseau, ainsi que des difficultés liées à la complexité d'intégration de ces solutions.

Apport du machine learning dans la détection des attaques

L'IA peut aujourd'hui aider les ingénieurs à détecter des menaces en entraînant un agent capable de détecter et de réagir automatiquement — bloquer ou autoriser un flux. L'année dernière, j'ai rédigé un article sur la création d'agents IA au niveau des routeurs Cisco pour détecter des événements BGP, notamment le BGP hijacking. L'utilisation de l'IA peut donc aider les spécialistes à mieux protéger leur infrastructure.

Blocage automatique par l'IA

Pas systématiquement. Un blocage automatique permanent n'est pas toujours souhaitable. On peut envisager un blocage temporaire (une à deux heures), qui peut être progressivement allongé si les mêmes adresses IP renouvellent leurs tentatives, puisque les attaquants changent souvent d'adresse IP.

Fonctionnalités essentielles d'une solution IA

Une solution IA efficace devrait passer par une phase d'entraînement, puis de test, avant la mise en production. Elle pourrait inclure un tableau de bord unifié avec la géolocalisation des IP, un système d'alertes par e-mail ou SMS, l'intégration via des API et la gestion des politiques de sécurité.

B.2 Entretien 2 — Soukaina Nafia

Informations sur l'interlocuteur

Nom et prénom	Soukaina Nafia
Poste / Fonction	Consultante en cybersécurité chez CY-CLIC Mentore en cybersécurité chez Connected Canadians Étudiante en cybersécurité à Polytechnique Montréal
Entreprise	CY-CLIC / SECOM
Secteur	Cybersécurité
Date de l'entretien	08 juin 2026
Lieu / Format	Microsoft Teams

Questions et réponses

Présentation et prise de contact

Bonjour, je suis ravie de vous accueillir aujourd'hui. Je m'appelle Khadija et nous allons avoir un échange très intéressant autour de l'intervention de l'intelligence artificielle dans la détection du trafic malveillant. Pourriez-vous nous parler brièvement de votre parcours et de votre rôle actuel dans la gestion des réseaux ?

Réponse : Bonjour, merci de m'avoir invitée. Mon nom est Soukaina Nafia. Je suis étudiante au baccalauréat en cybersécurité à Polytechnique Montréal. En parallèle, je travaille chez SECOM en tant que consultante et formatrice en prévention de la fraude et en cybersécurité. Mon rôle touche principalement la sensibilisation, l'analyse des risques, les bonnes pratiques de sécurité et l'accompagnement d'organisations souhaitant mieux protéger leurs données. Je fais également du bénévolat chez Connected Canadians, où j'aide à sécuriser un petit réseau informatique composé d'une vingtaine d'ordinateurs. Dans mon programme à Polytechnique, nous travaillons beaucoup en laboratoire avec des machines virtuelles, notamment sous Linux. Dans mon cours CR470 consacré aux tests d'intrusion, les travaux pratiques demandent d'utiliser des environnements virtuels comme Linux, Windows 10 et Metasploitable 2 pour effectuer de la reconnaissance, du balayage réseau et de l'analyse de vulnérabilités.

Principaux défis quotidiens

Dans une petite organisation, le plus grand défi est souvent le manque de ressources. Il n'y a pas toujours une personne dédiée à temps plein à la sécurité informatique. Les principaux défis sont la gestion des mots de passe, les mises à jour, le contrôle des accès, la protection des postes de travail et l'encadrement des accès à distance par VPN. Il faut également veiller à appliquer le principe du moindre privilège.

Comportements inhabituels observés

Dans un petit réseau, une anomalie typique peut se manifester par une latence élevée ou une connexion instable. Avant de conclure à une attaque, il faut examiner les causes

les plus simples : surcharge de la connexion Internet, problème de routeur, appareil consommant beaucoup de bande passante, mise à jour massive ou mauvaise configuration.

Apport du machine learning dans la détection des attaques

Je pense que l'IA peut être très utile pour détecter des anomalies plus rapidement, surtout lorsqu'il y a beaucoup de données à analyser. Un poste communiquant soudainement avec plusieurs adresses externes, un volume de trafic anormal ou des tentatives de connexion répétées peuvent être plus facilement détectés. Cependant, je ne la verrais pas comme une solution autonome d'emblée : l'IA devrait d'abord être un outil d'aide à la décision, capable de prioriser les alertes et d'expliquer pourquoi un comportement semble anormal.

Blocage automatique par l'IA

Oui, mais avec beaucoup de prudence. L'IA peut être utile pour recommander le blocage d'une adresse IP ou générer une alerte prioritaire. En revanche, je serais plus réservée concernant les décisions automatiques — comme le blocage d'un routeur ou la coupure d'un accès VPN — car une mauvaise décision pourrait bloquer tout un environnement. La meilleure approche serait progressive : d'abord une phase de détection et de signalement, puis une automatisation avec validation humaine, et enfin une autorisation uniquement pour des scénarios très bien définis et à faible risque.

Fonctionnalités essentielles d'une solution IA

Une solution IA efficace en cybersécurité devrait inclure : une détection d'anomalies en temps réel, un système de corrélation des événements, une priorisation intelligente des alertes, des explications claires des décisions de l'IA pour faciliter la prise de décision, et un contrôle humain maintenu dans la boucle pour les actions sensibles.

B.3 Entretien 3 — Oussama

Informations sur l'interlocuteur

Nom et prénom	Oussama
Poste / Fonction	Étudiant en 2 ^e année — Génie informatique — ENSA Kénitra
Entreprise	—
Secteur	Cybersécurité (contexte académique)
Date de l'entretien	28 avril 2026
Lieu / Format	Microsoft Teams

Questions et réponses

Présentation et architecture réseau utilisée

Je m'appelle Oussama. Je suis étudiant en deuxième année en génie informatique. Au quotidien, j'utilise principalement le réseau Wi-Fi de l'école pour accéder aux plateformes pédagogiques, effectuer des recherches, travailler sur GitHub et communiquer avec mon groupe de projet. En dehors des cours, j'utilise aussi parfois un petit environnement local avec Linux, Docker ou des machines virtuelles pour tester des applications web ou des scripts réseau.

Principaux défis liés à l'infrastructure réseau

Le problème principal est la disponibilité et la stabilité du réseau. Aux heures de pointe, lorsque de nombreux étudiants sont connectés, la connexion devient très lente. Parfois, le signal Wi-Fi est bon mais les pages ne chargent pas, ou les plateformes pédagogiques répondent très lentement.

Expérience récente d'anomalie réseau

Récemment, pendant un travail de groupe, nous devions envoyer un fichier et accéder à une plateforme en ligne, mais la connexion était très lente. Le Wi-Fi était connecté, mais les requêtes mettaient beaucoup de temps à répondre, provoquant des erreurs de type time-out. Le problème semblait général, touchant plusieurs services en même temps, comme si une saturation ou une anomalie affectait le trafic réseau.

Causes probables des perturbations furtives

Plusieurs causes sont possibles : la surcharge du réseau due à un grand nombre d'utilisateurs connectés, mais aussi des machines générant un trafic excessif à cause de téléchargements lourds, de mises à jour automatiques ou d'un malware. Dans un contexte de sécurité, une machine compromise pourrait lancer des scans de ports, envoyer de nombreuses requêtes ou tenter de découvrir les services ouverts sur le réseau.

Différencier une panne d'une attaque

À mon niveau, je ne peux pas le confirmer avec certitude, car je n'ai pas accès aux journaux des routeurs ou des pare-feux. Je peux néanmoins effectuer quelques vérifications simples : vérifier si le problème concerne uniquement mon poste ou plusieurs étudiants, faire un ping vers la passerelle pour mesurer la latence et les pertes de paquets.

Intégration de l'IA pour détecter les attaques en temps réel

Je pense que c'est pertinent, car un modèle d'IA pourrait analyser des volumes de données bien plus importants qu'un humain. Il pourrait détecter des patterns anormaux comme un nombre inhabituel de connexions, des scans de ports, des tentatives répétées vers plusieurs services ou une hausse brutale du trafic. L'avantage est sa réactivité, surtout si le modèle est entraîné sur des comportements normaux et malveillants. Il faut cependant veiller à limiter les faux positifs.

Conditions d'adoption d'une solution IA

Oui, je serais prêt à l'utiliser si elle améliore la stabilité du réseau et permet une détection rapide des anomalies. Les fonctionnalités importantes seraient une analyse en temps réel, des alertes claires et un tableau de bord simple. Mes principales craintes concernent la confidentialité des données, le risque de faux positifs et le fait que l'IA puisse prendre des décisions automatiques sans contrôle humain.

B.4 Entretien 4 — Ouzizi Amine

Informations sur l'interlocuteur

Nom et prénom	Ouzizi Amine
Poste / Fonction	Étudiant en 2 ^e année du cycle ingénieur — Réseaux et Systèmes de Télécommunication, option Sécurité des Réseaux
Entreprise	—
Secteur	Cybersécurité (contexte académique)
Date de l'entretien	28 avril 2026
Lieu / Format	Microsoft Teams

Questions et réponses

Présentation et architecture réseau utilisée

Je m'appelle Ouzizi Amine, étudiant en deuxième année du cycle ingénieur en réseaux et systèmes de télécommunication, option sécurité des réseaux et des systèmes d'information. J'utilise le réseau Wi-Fi de l'école tous les jours et je fais souvent tourner des environnements de test locaux sur mon poste Windows/Ubuntu pour avancer sur mes projets académiques et professionnels.

Expérience récente d'anomalie réseau

La semaine dernière, mon serveur web local plantait en boucle. En consultant les journaux de mon système, j'ai constaté des centaines de requêtes incomplètes sur différents ports, provenant d'adresses IP appartenant au réseau étudiant.

Causes probables des perturbations furtives

Souvent, ce sont des scripts automatisés ou des malwares installés à l'insu d'un autre étudiant sur son PC, qui tentent de trouver d'autres machines vulnérables sur le réseau local.

Intégration de l'IA pour détecter les attaques

Ce serait très utile. L'IA pourrait comprendre mon comportement habituel en tant que développeur sur le réseau et bloquer uniquement ce qui constitue réellement un trafic de reconnaissance malveillant.

Pertinence de l'IA pour intervenir sur l'infrastructure

Oui, surtout pour la modification dynamique. Si l'IA change le port de mon serveur de test dès qu'il est ciblé par un scan, cela me protège sans interrompre la connexion avec mes collègues de travaux pratiques.

Bilan et disposition à adopter la solution

Si ce système intelligent et autonome, garantissant un faible taux d'erreur, était déployé, je l'adopterais sans hésitation. Cela sécuriserait mes projets locaux sans effort supplémentaire de ma part.

B.5 Entretien 5 — Fahd

Informations sur l'interlocuteur

Nom et prénom	Fahd
Poste / Fonction	Étudiant en 1 ^{re} année du cycle ingénieur en informatique
Entreprise	—
Secteur	Cybersécurité (contexte académique)
Date de l'entretien	28 avril 2026
Lieu / Format	Microsoft Teams

Questions et réponses

Présentation et architecture réseau utilisée

Je m'appelle Fahd. Je suis étudiant en première année du cycle ingénieur en informatique. Je me connecte au réseau de l'école avec mon PC personnel pour suivre les cours et réaliser des travaux pratiques qui nécessitent souvent des communications multi-ports avec des machines virtuelles.

Principaux défis liés à l'infrastructure réseau

Les déconnexions intempestives constituent le principal défi. Parfois, le réseau me déconnecte simplement parce que je lance un outil de diagnostic nécessaire à un projet. Le filtrage est très restrictif.

Craintes avant d'autoriser une IA à interagir avec le réseau

Le taux de faux positifs doit être très faible. Je ne veux plus être empêché de rendre un projet à cause d'une règle de sécurité mal calibrée.

Caractéristiques souhaitées d'une solution IA

Il faut avant tout de la transparence. Si mon flux est bloqué, je veux une notification claire expliquant exactement quelle caractéristique de mon trafic a conduit l'IA à prendre cette décision.

Limites des méthodes actuelles

Les faux positifs, principalement. Les règles actuelles ne tiennent absolument pas compte du contexte académique de mes requêtes et me bloquent sans raison valable, ce qui est particulièrement problématique lors des rendus de projets.

Bilan et disposition à adopter la solution

Si cela met fin aux blocages injustifiés que nous subissons actuellement, je serais le premier à plaider pour son installation.

Appendix C

Prompts AI Générative

This appendix documents the generative AI prompts used throughout the project. Each prompt is categorized by its purpose: project scoping, code generation, report structuring, presentation creation, and review/refinement.

C.1 Prompt 1: Project Scoping and BRD Generation

Purpose: Generate the initial Business Requirements Document.

[j'ai un projet académique qui fait partie du module "intelligence artificielle ", ce projet consiste à travailler sur une topologie d'un réseau local qui intègrent plusieurs postes, l'un de ces postes est un potentiel attaquant de notre réseau et on a une intelligence artificielle qu'on va intégrer sur ce réseau , cette IA fait objet du coeur de notre projet , car on doit nous-même créer ce modèle et l'entraîner à partir des datasets spécifiques à un type d'attaque , et puis le tester , pour obtenir une intelligence artificielle capable de détecter le comportement suspect de cet cyber-attaquant et pouvoir intervenir pour le bloquer , le prof nous a détaillé en terme de notre cahier de charge ce qui suit : OUTPUT : L'IA doit être capable d'analyser le réseau , reporter d'une manière continue les indicateurs du réseau (Dashboard), détecter l'attaque, identifier le profil de l'attaquant , déclencher une alerte ou même une intervention , bloquer les accès d'une manière automatique. INPUT : ce modèle doit s'entraîner sur des datasets qui contiennent les infos du réseau (flux , adresses ip et adresses mac , subnets , adresse réseau et mask ... etc) pour pouvoir être capable de générer ce qui est attendue de lui dans la partie du output. Le prof nous a défini 4 étapes majeures qu'on doit suivre parfaitement à la lettre au sein de notre projet sur lesquelles le projet est noté: 1- la conception 2-l'implémentation 3-le test 4-la validation Maintenant j'ai besoin que tu me donnes tous les outils et les détails techniques dont j'aurai besoin dans chacune de ces 4 étapes en ordre , la stratégie adéquate à suivre , une roadmap claire pour le déroulement du projet , tous les types de défis qu'on peut rencontré tout au long de notre travail , des conseils pertinents pour bien s'orienter (par exemple type d'attaque qu'on doit choisir.... etc) Mets en tête qu'on travaille sur ce projet comme binôme et la note du projet est primordial pour ce module , donc on est sensé faire vraiment un travail de pro. Pour cela et tenant en considération tout ce qui précède , je m'attends de ta part une réponse optimale , claire , détaillée , méthodique et qui répond parfaitement à toutes ces questions que j'ai posé et qui aussi satisfait nos besoins techniques. C'est parti !]

[According to the following interview report we have done with expert people, generate a second version of a BRD for our project, suggest what exact type of attack we gonna work on depending on the interviews]

C.2 Prompt 2: ML Pipeline Code Generation

Purpose: Generate the data preprocessing and model training pipeline.

[Now that we have done the conception phase, our next plan is to prepare for the implementation section that consists of preparing the data and training our modules according to the conception results. ur task is to help me organize this section, and provide me with a suggested structure of the pipeline to follow]

C.3 Prompt 3: Report Structuring and Chapter Generation

Purpose: Structure and draft the final report following the CRISP-DM framework.

[Now that we have successfully completed all the technical phases, the main remaining task is to document everything we have done, our next plan is to write a report that documents all the three big sections from conception to implementation to validation and testing our models, therefor i ask ur help to suggest a report structure that fits our work]

C.4 Prompt 4: Presentation Deck Creation

Purpose: Create the defense presentation with 19 slides.

[this is a quick task to complete, i want you to generate a simple yet elaborated presentation in html that covers the main point of our report and following ur own suggestion structure]]

C.5 Prompt 5: Code Review and Refinement

Purpose: Review and clean the repository codebase.

[please write a clean code that follows our previous structure we have settled to prepare our data and train the models.]

[i have added some changes to the code, review it and suggest refinements]

Appendix D

Code Source

This appendix provides a reference to the project's source code structure and a drive link with the code source. Visit the [code source drive folder](#) accompanying `code_source_python.zip` archive, we provided a `readme.md` file that explain the folder structure.

Or you can reach the complete source code available in the project repository. Visit the [github repo](#).

D.1 Project Directory Structure

```
Portscan_IDS/
  detection/src/          ML pipeline and inference
    cicids2017_pipeline.py Full CRISP-DM pipeline
    config.py             Local config (feature list)
    data_preprocessing.py  IQR/outlier analysis
    evaluate_model.py      Model evaluation + metrics
    feature_selection.py   Feature selection scripts
    modeltrain.py          Legacy training script
    predict.py             CLI inference script
    preprocessing_utils.py Shared preprocessing (IQR, log1p)
    retrain_11features.py  Retraining script (canonical)
  bridge/                 Integration bridge
    bridge.py              ML prediction bridge
    nmap_parser.py         Nmap XML to feature CSV
    test_pipeline.py       End-to-end test suite
  dashboard/              SOC monitoring dashboard
    app.py                 Flask + Socket.IO backend
    templates/             HTML frontend
    static/                CSS/JS assets
  deception/              Active defense modules
    core_deception.py      Honeypot + phantom network
    network_mutator.py     MTD port rotation
    traffic_shaper.py      Packet-level mutation
    monitor_interface.py   IP blacklisting manager
  capture/                Network capture utilities
    fetch_and_align_datasets.py Dataset download
```

scans/	10 Nmap scan XMLs
demo/	Demo attack scripts
kamli_attack.sh	Attack simulation
kali_custom_scanner.py	Custom scanner
ubuntu_setup.sh	Target setup
models/saved/	Trained model artifacts
rf_model.pkl	Random Forest (100 trees)
xgb_model.pkl	XGBoost (100 estimators)
isolation_forest.pkl	Isolation Forest
scaler.pkl	StandardScaler
evaluation_metrics.json	Final metrics
preprocessing.json	Preprocessing parameters
config.py	Project-level configuration
run_demo.py	Unified demo entry point
requirements.txt	Python dependencies

Appendix E

Taux d'Écriture AI

AI Writing Rate Analysis

The AI writing rate was verified using an external detection tool (e.g., GPTZero, Originality.ai, Turnitin AI Detection, or equivalent).

Property	Value
Tool used	[TOOL_NAME_HERE]
Analysis date	[DATE_HERE]
AI-generated percentage	[XX]%
Human-written percentage	[XX]%

Note: There is no minimum or maximum required AI writing rate for this project (as specified by the professor). This appendix is provided for transparency regarding the use of generative AI tools in the preparation of this report.

Appendix F

Taux de Plagiat

Plagiarism Rate Analysis

The plagiarism rate was verified using an external detection tool (e.g., Turnitin, Compilatio, Urkund, or equivalent).

Property	Value
Tool used	[TOOL_NAME_HERE]
Analysis date	[DATE_HERE]
Overall similarity	[XX]%

Note: There is no minimum or maximum required plagiarism rate for this project (as specified by the professor). This appendix is provided for academic integrity verification purposes.